Ex. No. 9	Update Information in Google Firestore Database
Date of Exercise	01 November 2026
Github Code Link(Keep as public)	https://github.com/henrysam/FlutterLaboratory

Aim

To develop a Flutter application that updates existing student information in Google Cloud Firestore based on user input and displays the updated information on the screen.

Description

Google Cloud Firestore is a cloud-hosted NoSQL database that stores information in collections and documents. In this experiment, a Flutter application named Student Update is developed to modify student information already stored in the 'students' collection. The user enters a student's register number and selects the Find Student button. The application searches Firestore using the where() method with an equality condition on the registerNumber field. If a matching document is found, its document ID and current student details are retrieved and displayed in editable text fields. The user can modify the student's name, department and email address. When the Update Student button is pressed, the application uses the Firestore document reference and update() method to modify the existing document. The register number is used to identify the record and is not changed. A success message is displayed after the update operation completes. Firebase is initialized using Firebase.initializeApp() and the configuration generated by the FlutterFire CLI. The application was run in Google Chrome, and the successful update operation was verified by the displayed confirmation message.

Program

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
```

```
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Student Update',
      theme: ThemeData(
        primarySwatch: Colors.indigo,
        useMaterial3: true,
      ),
      home: const UpdatePage(),
    );
  }
}

class UpdatePage extends StatefulWidget {
  const UpdatePage({super.key});

  @override
  State<UpdatePage> createState() => _UpdatePageState();
}

class _UpdatePageState extends State<UpdatePage> {
  final registerController = TextEditingController();
  final nameController = TextEditingController();
  final departmentController = TextEditingController();
  final emailController = TextEditingController();

  final CollectionReference students =
      FirebaseFirestore.instance.collection('students');

  String? documentId;
  bool isLoading = false;

  Future<void> searchStudent() async {
    final registerNumber = registerController.text.trim();

    if (registerNumber.isEmpty) return;

    setState(() {
      isLoading = true;
      documentId = null;
    });

    try {
      final query = await students
          .where('registerNumber', isEqualTo: registerNumber)
          .get();
```

```
    if (query.docs.isNotEmpty) {
      final doc = query.docs.first;
      final data = doc.data() as Map<String, dynamic>;

      setState(() {
        documentId = doc.id;
        nameController.text = data['name'] ?? '';
        departmentController.text = data['department'] ?? '';
        emailController.text = data['email'] ?? '';
        isLoading = false;
      });
    } else {
      setState(() {
        nameController.clear();
        departmentController.clear();
        emailController.clear();
        isLoading = false;
      });

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Student not found')),
      );
    }
  } catch (e) {
    setState(() => isLoading = false);

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Search failed: $e')),
    );
  }
}

Future<void> updateStudent() async {
  if (documentId == null) return;

  try {
    await students.doc(documentId).update({
      'name': nameController.text.trim(),
      'department': departmentController.text.trim(),
      'email': emailController.text.trim(),
    });

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Student updated successfully!'),
      ),
    );

    setState(() {});
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Update failed: $e')),
    );
  }
}
```

```
@override
void dispose() {
  registerController.dispose();
  nameController.dispose();
  departmentController.dispose();
  emailController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Update Student Information'),
      backgroundColor: Colors.indigo,
      foregroundColor: Colors.white,
    ),
    body: Padding(
      padding: const EdgeInsets.all(20),
      child: SingleChildScrollView(
        child: Column(
          children: [
            TextField(
              controller: registerController,
              decoration: const InputDecoration(
                labelText: 'Enter Register Number',
                border: OutlineInputBorder(),
                prefixIcon: Icon(Icons.search),
              ),
            ),

            const SizedBox(height: 12),

            ElevatedButton.icon(
              onPressed: searchStudent,
              icon: const Icon(Icons.search),
              label: const Text('Find Student'),
            ),

            const SizedBox(height: 20),

            if (isLoading)
              const CircularProgressIndicator(),

            if (documentId != null) ...[
              const Text(
                'Edit Student Details',
                style: TextStyle(
                  fontSize: 20,
                  fontWeight: FontWeight.bold,
                ),
              ),

              const SizedBox(height: 15),
```

```
            TextField(
              controller: nameController,
              decoration: const InputDecoration(
                labelText: 'Student Name',
                border: OutlineInputBorder(),
              ),
            ),

            const SizedBox(height: 12),

            TextField(
              controller: departmentController,
              decoration: const InputDecoration(
                labelText: 'Department',
                border: OutlineInputBorder(),
              ),
            ),

            const SizedBox(height: 12),

            TextField(
              controller: emailController,
              decoration: const InputDecoration(
                labelText: 'Email',
                border: OutlineInputBorder(),
              ),
            ),

            const SizedBox(height: 20),

            ElevatedButton.icon(
              onPressed: updateStudent,
              icon: const Icon(Icons.update),
              label: const Text('Update Student'),
            ),
          ],
        ],
      ),
    ),
  ),
 );
 }
}
```

Output Screenshots

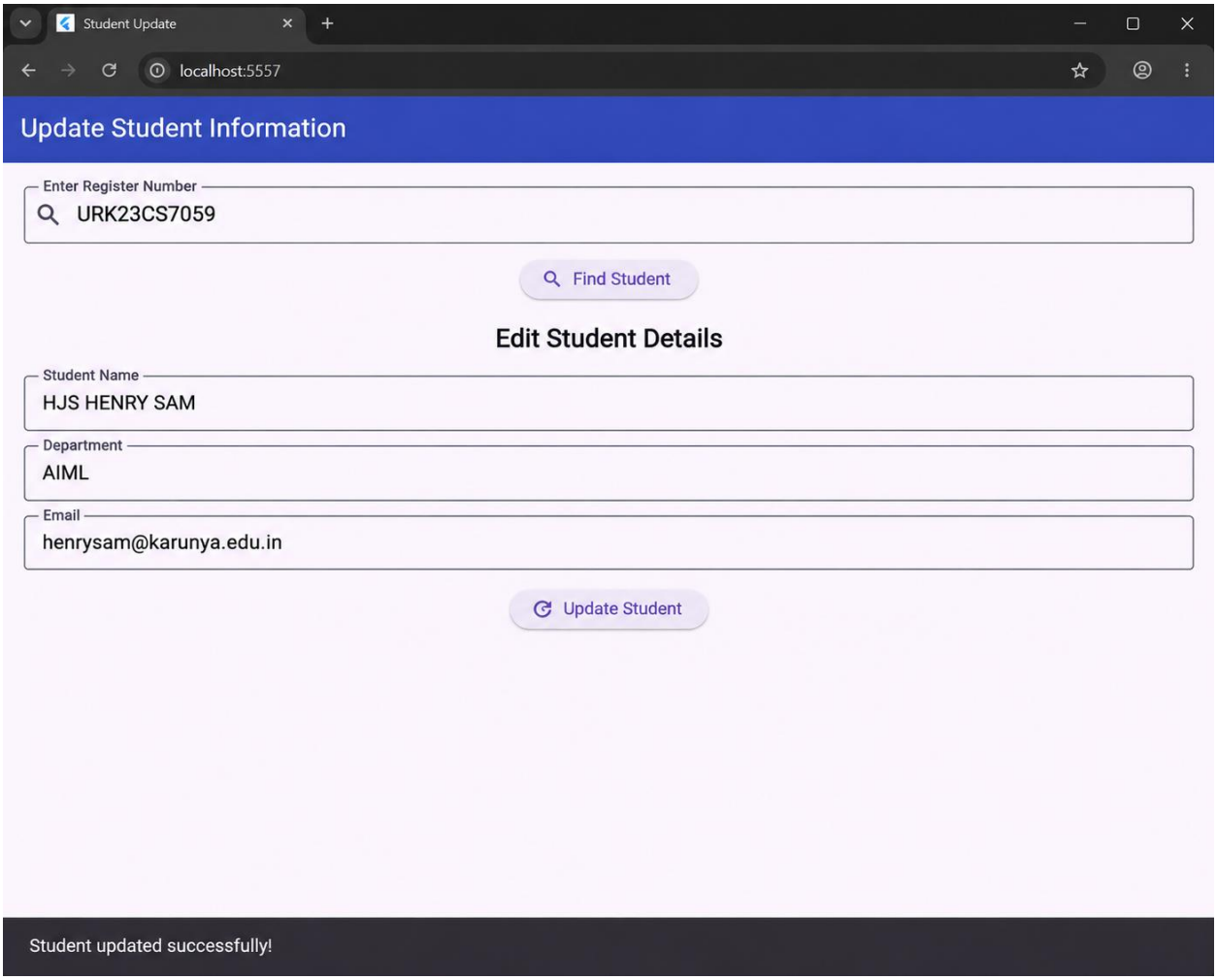


Figure 1: Student Update application displaying the retrieved student details and the successful update confirmation message.

// Output of the program

Result

Thus, the Flutter application for updating student information in Google Cloud Firestore was successfully developed and executed. The application retrieved an existing student record using the register number, displayed the details in editable fields and updated the existing Firestore document using the update() method. The successful update confirmation was displayed on the application screen.